Table of contents

Question of Interest:

How accurately can the hours of social media and gaming predict the day-to-day mental health outcomes of individuals?

Target Variable: Daily Mood

Original Dataset before we made any changes. As you all can see, the dataset is a bit unorganized since there could be duplicates, null values, too many columns, wrong datatype, etc. Below we will be cleaning it so we can get the resources we will need.

In [ ]:
```
#Importing the classes
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn import tree
from sklearn.metrics import accuracy_score, confusion_matrix, f1_score, mean_squared_error, ConfusionMatrixDisplay, r
```

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.preprocessing import OrdinalEncoder
```

In [ ]:
```
df = pd.read_csv("/Users/sana/Desktop/DTSC 2301/Final Project/Final_project (1).csv")
df.head()
```

Out[ ]:

	Timestamp	Age Range	Gender	Are you a student?	How many hours per day do you spend on social media?	Which platforms do you use the most?	How often do you check social media?	What do you mostly do on social media?	Do you play video games?	If yes, how many hours per day do you game?	...	I feel addicted to social media or gaming	On average, how would you rate your daily mood?	r h s
0	4/13/2026 15:51:41	18-21	Male	Yes	3-5 hours	Instagram, Snapchat	A few times a day	Scroll/watch content	Yes	Less than 1 hour	...	4	Neutral	I
1	4/13/2026 15:55:27	22-25	Male	Yes	More than 5 hours	Instagram, TikTok, Snapchat, Youtube	A few times a day	Chat with friends	No	Less than 1 hour	...	2	Neutral	I
2	4/13/2026 15:55:51	18-21	Female	Yes	3-5 hours	Instagram, Snapchat, LinkedIn	Every hour	Chat with friends	No	Less than 1 hour	...	2	Good	I
3	4/13/2026 15:57:05	18-21	Female	Yes	More than 5 hours	Instagram, Snapchat, Youtube	Every hour	Chat with friends	No	NaN	...	1	Good	I
4	4/13/2026 16:00:15	18-21	Male	No	More than 5 hours	Instagram, TikTok, Twitter/X, Youtube	Constantly	Scroll/watch content	Yes	3-5 hours	...	3	Good	I

5 rows × 30 columns

Part 1: Data Cleaning and Organization

We are starting to clean up the data a little bit. Here is our process:

1. Drop the unnecessary columns that do not help out as much.
2. We will rename the columns for more readibility.
3. We will check for any null and duplicate values and remove them if we find any.
4. We will use ordinal encoding to temporarily convert Daily Mood to a numerical variable to aid us in creating the prelim visual aids.
5. After all our cleaning is done, we will double check if there is any cleaning needed.

In [ ]:
```
#Removing the Unnecessary Columns
df = df.drop(columns=[' How does social media affect your mood?', 'How does gaming affect your stress levels?'], axis
df = df.drop(columns=['What do you mostly do on social media?', 'What type of games do you play most?'], axis=1)
df = df.drop(columns=['How often do you interact (likes/comments/messages)?',
                      'Do you skip responsibilities for gaming/social media?'], axis=1)
df = df.drop(columns=['Do you check your phone immediately after waking up?', 'Do you play games late at night (past
df = df.drop(columns=['Do you lose track of time while scrolling or gaming?', 'Timestamp'], axis=1)
df = df.drop(columns=['Are you a student?', 'Which platforms do you use the most?', ' How often do you check social m
                     'I feel happier after gaming', 'I feel frustrated or angry after gaming',
                      ' How many hours of sleep do you get?', ' How often do you feel stressed?'], axis=1)
```

In [ ]:
```
#Renaming the Column Names
df = df.rename(columns={'How many hours per day do you spend on social media?': 'Social Media Hours',
                        'If yes, how many hours per day do you game?': 'Video Game Hours',
                        'I feel anxious after using social media': 'Anxiety Levels',
                        'I compare myself to others on social media': 'Comparision',
                        'Social media affects my self-esteem': 'Self Esteem',
                        'I use social media or gaming to escape stress': 'Using for Relief',
                        'I feel lonely even when using social media': 'Lonliness',
                        'My sleep is affected by social media/gaming': 'Sleep Affected',
                        'I feel addicted to social media or gaming': 'Addiction Levels',
                        'On average, how would you rate your daily mood?': 'Daily Mood',
                        'My sleep is affected by social media/gaming': 'Sleep Affected',
                        ' Do you play video games?': 'Gamer?'
                        })
```

In [ ]:
```
#Check for null values
df.isnull().sum()
```

Out[ ]:
```
Age Range             0
Gender                0
Social Media Hours    0
Gamer?                0
Video Game Hours     29
Anxiety Levels        0
Comparision           0
Self Esteem           0
Using for Relief      3
Lonliness             1
Sleep Affected        1
Addiction Levels      0
Daily Mood            0
dtype: int64
```

In [ ]:
```
df
```

Out[ ]:

	Age Range	Gender	Social Media Hours	Gamer?	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	18-21	Male	3-5 hours	Yes	Less than 1 hour	2	2	3	3.0	3.0	3.0	4	Neutral
1	22-25	Male	More than 5 hours	No	Less than 1 hour	4	2	2	2.0	3.0	3.0	2	Neutral
2	18-21	Female	3-5 hours	No	Less than 1 hour	2	1	2	3.0	1.0	4.0	2	Good
3	18-21	Female	More than 5 hours	No	NaN	4	5	4	NaN	3.0	1.0	1	Good
4	18-21	Male	More than 5 hours	Yes	3-5 hours	1	2	1	3.0	2.0	3.0	3	Good
...	...	...	...	...	...	...	...	...	...	...	...	...	...
95	18-21	Male	1-3 hours	Yes	1-3 hours	4	4	5	4.0	5.0	4.0	5	Very Good
96	26+	Female	1-3 hours	No	Less than 1 hour	5	5	2	2.0	5.0	3.0	4	Very Good
97	22-25	Female	More than 5 hours	Maybe	1-3 hours	3	4	5	4.0	5.0	4.0	4	Very Good
98	Under 18	Female	1-3 hours	Maybe	More than 5 hours	3	3	1	4.0	4.0	5.0	3	Neutral
99	22-25	Male	3-5 hours	Yes	More than 5 hours	5	5	5	5.0	5.0	5.0	5	Very low

100 rows × 13 columns

I am trying ordinal encoding for the first time to make the line graphs.

In [ ]:
```
df['Daily Mood'].unique()
```

Out[ ]:
```
array(['Neutral', 'Good', 'low', 'Very Good', 'Very low'], dtype=object)
```

In [ ]:
```
#Using ordinal encoding to convert Daily Mood from Categorical vars to numerical vars

enc = OrdinalEncoder(categories=[['Very low','low','Neutral','Good','Very Good']])
enc.fit(df[['Daily Mood']])
df['Daily Mood'] = enc.transform(df[['Daily Mood']])
df.head()
```

Out[ ]:

	Age Range	Gender	Social Media Hours	Gamer?	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	18-21	Male	3-5 hours	Yes	Less than 1 hour	2	2	3	3.0	3.0	3.0	4	2.0
1	22-25	Male	More than 5 hours	No	Less than 1 hour	4	2	2	2.0	3.0	3.0	2	2.0
2	18-21	Female	3-5 hours	No	Less than 1 hour	2	1	2	3.0	1.0	4.0	2	3.0
3	18-21	Female	More than 5 hours	No	NaN	4	5	4	NaN	3.0	1.0	1	3.0
4	18-21	Male	More than 5 hours	Yes	3-5 hours	1	2	1	3.0	2.0	3.0	3	3.0

Checking if there is any cleaning left.

In [ ]:
```
#Checking if there is anything else to clean up
df.dtypes
```

Out[ ]:
```
Age Range              object
Gender                 object
Social Media Hours     object
Gamer?                 object
Video Game Hours       object
Anxiety Levels          int64
Comparision             int64
Self Esteem             int64
Using for Relief      float64
Lonliness             float64
Sleep Affected        float64
Addiction Levels        int64
Daily Mood            float64
dtype: object
```

In [ ]:
```
df.duplicated()
```

Out[ ]:
```
0     False
1     False
2     False
3     False
4     False
      ...
95    False
96    False
97    False
98    False
99    False
Length: 100, dtype: bool
```

In [ ]:
```
df.isnull().sum()
```

Out[ ]:
```
Age Range              0
Gender                 0
Social Media Hours     0
Gamer?                 0
Video Game Hours      29
Anxiety Levels         0
Comparision            0
Self Esteem            0
Using for Relief       3
Lonliness              1
Sleep Affected         1
Addiction Levels       0
Daily Mood             0
dtype: int64
```

Part 2: Preliminary Graphs

I picked a line graph both for Social Media and Gaming because I feel like using a line graph can clearly show the trend in both areas to showcase their Daily Mental Health effects. I used Label Encoding to temproarily convert Social Media and Gaming hours to numerical values to make the math more easier. I assigned the closest and the most fair integer to the categorical numbers.

Example:

Less than 1 hour --> 0.5

3-5 hours --> 4

Line Graph that explores r-ship btw Social Media Hours and Daily Mood

In [ ]:
```
df['Social Media Hours'].unique()
```

Out[ ]:
```
array(['3-5 hours', 'More than 5 hours', '1-3 hours', 'Less than 1 hour',
       'Yes'], dtype=object)
```

I used Ordinal Encoding to preserve the relationship between the values in our columns.

In [ ]:
```
#Changing SM hours to numerical
df = df[df['Social Media Hours'] != 'Yes']
```

```
sm_enc = OrdinalEncoder(categories=[['Less than 1 hour','1-3 hours','3-5 hours','More than 5 hours']])
sm_enc.fit(df[['Social Media Hours']])
df['Social Media Hours'] = sm_enc.transform(df[['Social Media Hours']])
df.head()
```

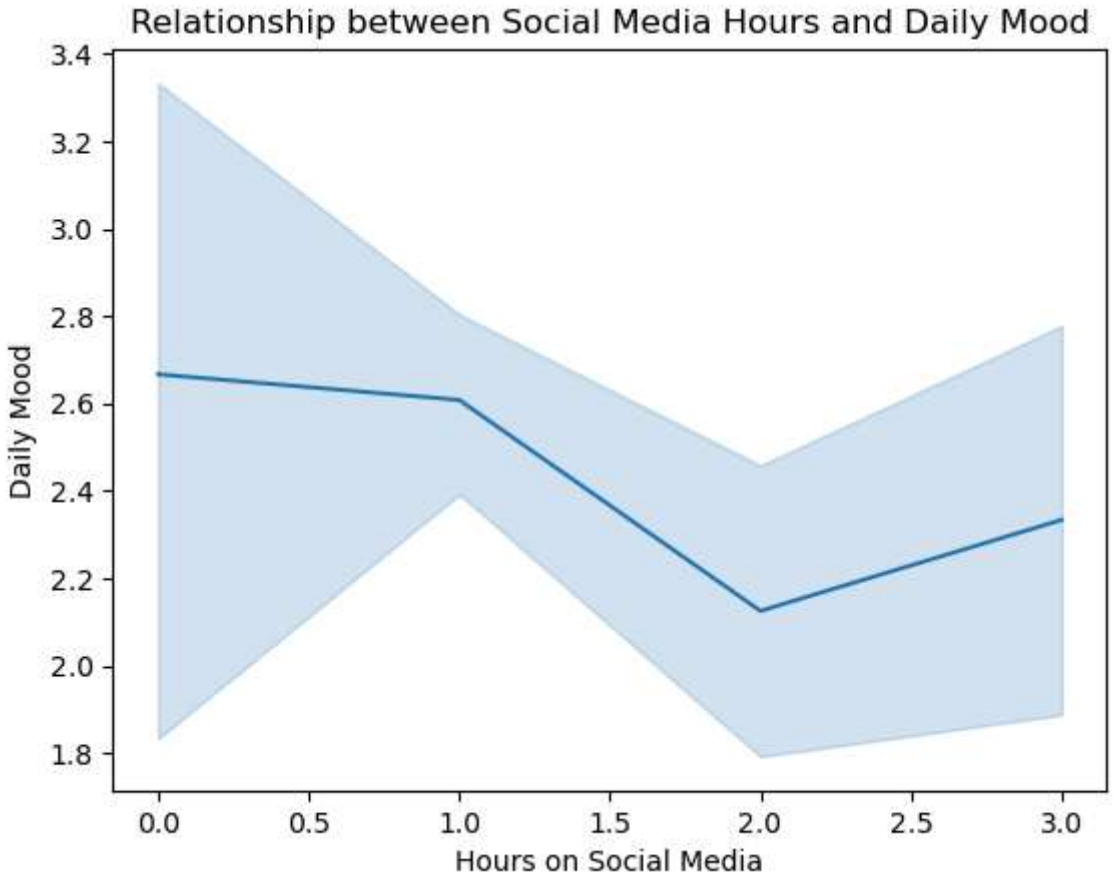
```
/var/folders/7k/zly6nxhd6_q260hyvbclvmym0000gn/T/ipykernel_25258/3960356865.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning
-a-view-versus-a-copy
  df['Social Media Hours'] = sm_enc.transform(df[['Social Media Hours']])
```

Out[ ]:

	Age Range	Gender	Social Media Hours	Gamer?	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	18-21	Male	2.0	Yes	Less than 1 hour	2	2	3	3.0	3.0	3.0	4	2.0
1	22-25	Male	3.0	No	Less than 1 hour	4	2	2	2.0	3.0	3.0	2	2.0
2	18-21	Female	2.0	No	Less than 1 hour	2	1	2	3.0	1.0	4.0	2	3.0
3	18-21	Female	3.0	No	NaN	4	5	4	NaN	3.0	1.0	1	3.0
4	18-21	Male	3.0	Yes	3-5 hours	1	2	1	3.0	2.0	3.0	3	3.0

In [ ]:

```
#Line Graph that explores r-ship btw Social Media Hours and Daily Mood
sns.lineplot(data=df, x="Social Media Hours", y="Daily Mood")
plt.xlabel("Hours on Social Media")
plt.ylabel("Daily Mood")
plt.title('Relationship between Social Media Hours and Daily Mood')

plt.show()
```



Line Graph that explores r-ship btw Gaming Hours and Daily Mood

In [ ]:
```
df['Video Game Hours'].unique()
```

Out[ ]:
```
array(['Less than 1 hour', nan, '3-5 hours', '1-3 hours',
       'More than 5 hours'], dtype=object)
```

In [ ]:
```
#Changing Gaming hours to numerical
# Remove rows where Video Game Hours is missing before fitting the encoder
df = df[df['Video Game Hours'].notna()]

gm_enc = OrdinalEncoder(categories=[['Less than 1 hour','1-3 hours','3-5 hours','More than 5 hours']])
gm_enc.fit(df[['Video Game Hours']])
```

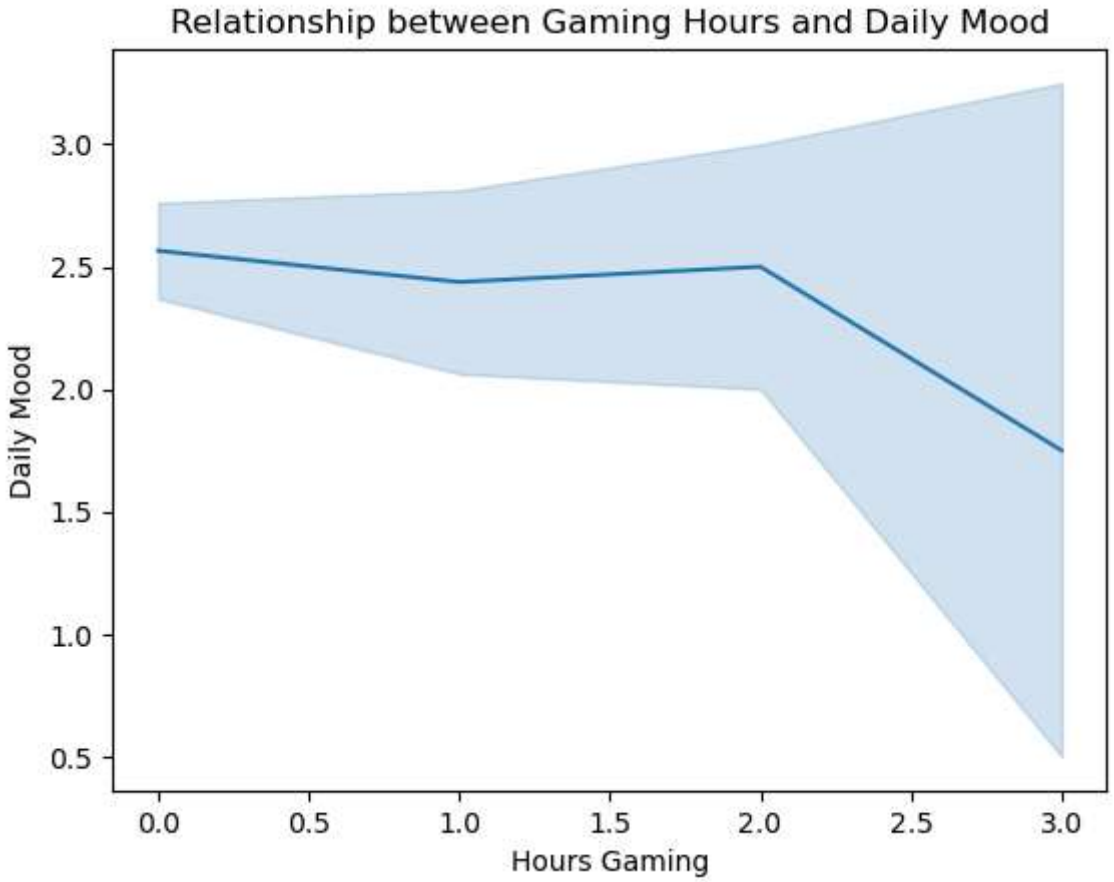
```
df['Video Game Hours'] = gm_enc.transform(df[['Video Game Hours']])
df.head()
```

Out[ ]:

	Age Range	Gender	Social Media Hours	Gamer?	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	18-21	Male	2.0	Yes	0.0	2	2	3	3.0	3.0	3.0	4	2.0
1	22-25	Male	3.0	No	0.0	4	2	2	2.0	3.0	3.0	2	2.0
2	18-21	Female	2.0	No	0.0	2	1	2	3.0	1.0	4.0	2	3.0
4	18-21	Male	3.0	Yes	2.0	1	2	1	3.0	2.0	3.0	3	3.0
5	18-21	Female	2.0	Maybe	0.0	2	3	2	4.0	2.0	1.0	2	2.0

In [ ]:

```
#Line Graph that explores r-ship btw Gaming Hours and Daily Mood
sns.lineplot(data=df, x="Video Game Hours", y="Daily Mood")
plt.xlabel("Hours Gaming")
plt.ylabel("Daily Mood")
plt.title('Relationship between Gaming Hours and Daily Mood')

plt.show()
```



Part 3: TRAIN TEST SPLIT

Model 1: SVM - Helps with seperating social media and gaming by introducing the hard margin.

Model 2: KNN - Uses Eucleadian distance to find the nearest neighbors. Can determine who can be more affected by gaming or social media.

Model 3: Linear Regression - Helps us predict data in numerical form by adding a line of best fit.

Getting started with the training set

In [ ]:
```
#Changing Gamer? to a Numerical Value
df['Gamer?'].unique()


gamer_map = { 'No': 0,
          'Maybe':1,
          'Yes':2,

       }
df['Gamer?'] = df['Gamer?'].map(gamer_map)
```

Gamer? is now numerical

0 = No

1 = Maybe

2 = Yes

In [ ]:
```
#Building the training set

df_subset = df[['Gamer?', 'Social Media Hours','Video Game Hours',
                'Anxiety Levels', 'Comparision',
                'Self Esteem', 'Using for Relief',
                'Lonliness', 'Sleep Affected', 'Addiction Levels',
                'Daily Mood']]


df_subset
```

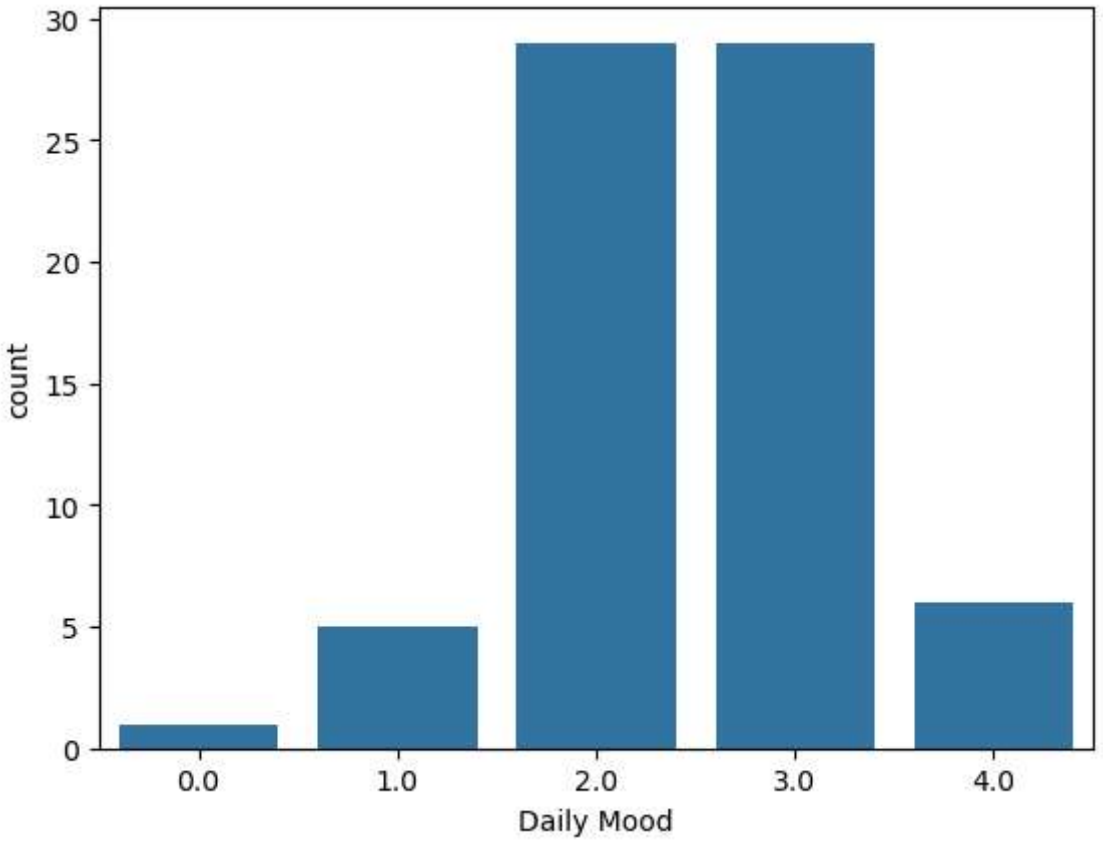
Out[ ]:

	Gamer?	Social Media Hours	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	2	2.0	0.0	2	2	3	3.0	3.0	3.0	4	2.0
1	0	3.0	0.0	4	2	2	2.0	3.0	3.0	2	2.0
2	0	2.0	0.0	2	1	2	3.0	1.0	4.0	2	3.0
4	2	3.0	2.0	1	2	1	3.0	2.0	3.0	3	3.0
5	1	2.0	0.0	2	3	2	4.0	2.0	1.0	2	2.0
...	...	...	...	...	...	...	...	...	...	...	...
95	2	1.0	1.0	4	4	5	4.0	5.0	4.0	5	4.0
96	0	1.0	0.0	5	5	2	2.0	5.0	3.0	4	4.0
97	1	3.0	1.0	3	4	5	4.0	5.0	4.0	4	4.0
98	1	1.0	3.0	3	3	1	4.0	4.0	5.0	3	2.0
99	2	2.0	3.0	5	5	5	5.0	5.0	5.0	5	0.0

70 rows × 11 columns

I created a countplot to understand how our labels are distributed. This will provide insight as to which metrics to use to determine how well our models perform as well as whether we need to stratify our data during our initial training and testing splits and cross validation.

In [ ]:
```
#Loading the countplot

sns.countplot(data=df_subset, x='Daily Mood')

plt.show()
```



We are going to do a train test split. We will make Daily Mood the predicted and everything else the predictors.

80% Train

20% Test

Todo List:

Hyper Parameter Tuning - Tuning the models before training them. This controls how the model learns not what it learns

In [ ]:
```
mood = []

for i in df_subset['Daily Mood']:
    if i == 0.0 or i == 1.0:
```

```
        mood.append('Bad')

    elif i == 2.0:
        mood.append('Neutral')

    else:
        mood.append('Good')
```

In [ ]:
```
#Choosing the Predictors and the Predicted

X = df_subset.drop(columns=['Daily Mood'])
y = mood
```

In [ ]:
```
#Executing the Train Test Split
# Ensure predictors X and target y are defined (in case the cell that created them wasn't run)
X = df_subset.drop(columns=['Daily Mood'])
y = mood

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=11, test_size=0.2, stratify=y)
```

Part 3.1: SVM MODEL

We chose the Support Vector Machine (SVM) because it will add a hard margin between the gamers and and people who only use social media. This will help us predict their daily mood in a more organized way.

Pros of SVM:

- More readable Graphs
- Can be efficient and effective in high dimensional spaces
- Uses a hard line to make the graphs variables easier to distinguish

Cons of SVM:

- Low Accuracy in very large datasets
- Outliers can influence boundaries

- SVM requires long training periods which can lead to inefficiency

In [ ]:
```
#SVM Importations
from sklearn.svm import SVC
from sklearn.datasets import make_blobs
from sklearn.inspection import DecisionBoundaryDisplay
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import roc_auc_score
```

In [ ]:
```
svc_model = SVC(class_weight="balanced")
```

In [ ]:
```
svc_model.fit(X_train, y_train)
```

Out[ ]:
```
SVC
SVC(class_weight='balanced')
```

With a score of 0.62, we can see that while it is accurate to a certain extent, there are things that could have been improved with the data so that we could have a higher score.

In [ ]:
```
svc_model.score(X_train, y_train)
```

Out[ ]:
```
0.625
```

SVM Evaluation on Test Set

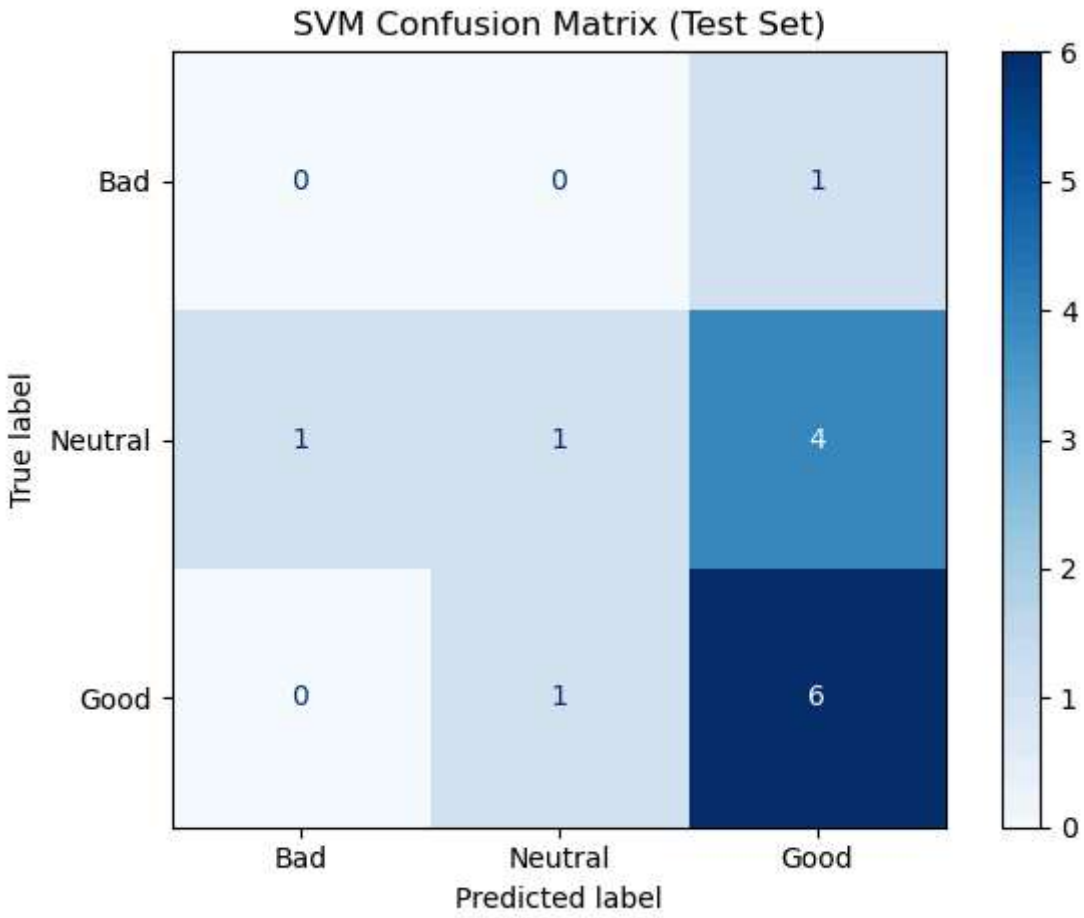
Now that the model is trained, we evaluate it on the **held-out test set**. We use accuracy, precision, recall, and F1-score. Recall is particularly important here — missing a person who is in a 'Bad' mood is more costly than a false alarm.

In [ ]:
```
## SVM Test Set Evaluation

# Ensure X_test_imputed exists (impute using training stats if needed)
if 'X_test_imputed' not in globals():
        # import only if not already available in the notebook
        if 'SimpleImputer' not in globals():
```

```
                from sklearn.impute import SimpleImputer
        imputer = SimpleImputer(strategy='median')
        imputer.fit(X_train)  # fit on training data only
        X_test_imputed = pd.DataFrame(imputer.transform(X_test),
                                      columns=X_test.columns,
                                      index=X_test.index)

svc_test_score = svc_model.score(X_test_imputed, y_test)
svc_train_score = svc_model.score(X_train, y_train)
print(f'SVM Train Accuracy: {svc_train_score:.3f}')
print(f'SVM Test  Accuracy: {svc_test_score:.3f}')

svc_preds = svc_model.predict(X_test_imputed)
print('\nClassification Report (SVM):')
print(classification_report(y_test, svc_preds))
```

```
SVM Train Accuracy: 0.625
SVM Test  Accuracy: 0.500

Classification Report (SVM):
              precision    recall  f1-score   support

         Bad       0.00      0.00      0.00         1
        Good       0.55      0.86      0.67         7
     Neutral       0.50      0.17      0.25         6

    accuracy                           0.50        14
   macro avg       0.35      0.34      0.31        14
weighted avg       0.49      0.50      0.44        14
```

In [ ]:
```
# Confusion Matrix – SVM
cm_svc = confusion_matrix(y_test, svc_preds, labels=['Bad', 'Neutral', 'Good'])
disp = ConfusionMatrixDisplay(confusion_matrix=cm_svc, display_labels=['Bad', 'Neutral', 'Good'])
disp.plot(cmap='Blues')
plt.title('SVM Confusion Matrix (Test Set)')
plt.tight_layout()
plt.show()
```



Part 3.2: KNN MODEL

For our next model, we will be using KNN (K Nearest Neighbors) to predict the daily mood. RMSE would be the best scoring method to use

KNN Pros

- No Assumptions
- Handles Multi Class Problems
- Easy to Interpret

KNN Cons

- Sensitive to Outliers
- Struggles with unbalanced datasets
- High Memory Usage

In [ ]:
```
#KNN Importations:
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

In [ ]:
```
knn = KNeighborsClassifier(n_neighbors=9)
```

In [ ]:
```
knn.fit(X_train, y_train)
```

Out[ ]:
```
KNeighborsClassifier(n_neighbors=9)
```

In [ ]:
```
from sklearn.impute import SimpleImputer

# Impute missing values in X_test using statistics from X_train, then predict
imputer = SimpleImputer(strategy='median')
imputer.fit(X_train)  # fit only on training data
X_test_imputed = pd.DataFrame(imputer.transform(X_test), columns=X_test.columns, index=X_test.index)


# Impute missing values in
```

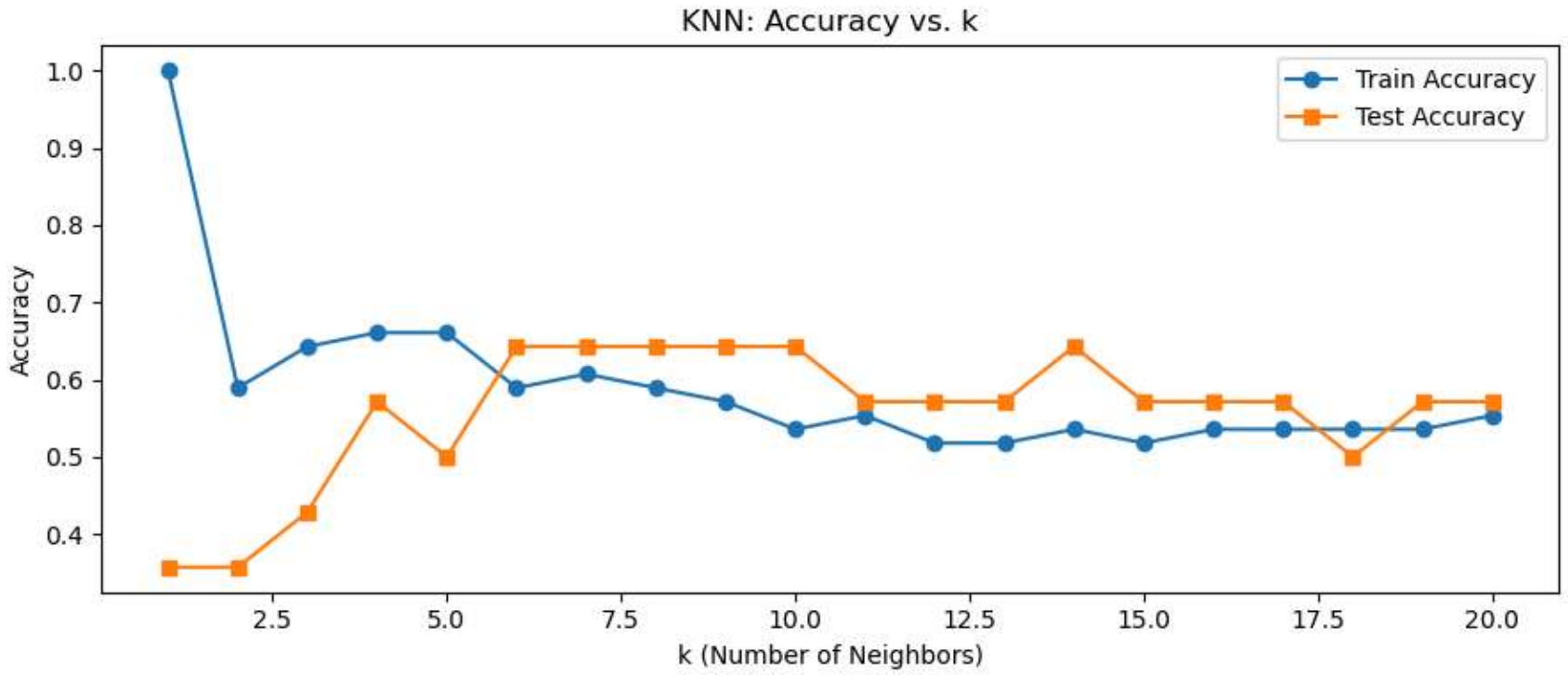
In [ ]:
```
# Evaluate KNN on the imputed test set (use X_test_imputed created earlier)
knn.score(X_test_imputed, y_test)
```

Out[ ]:
```
0.6428571428571429
```

KNN Hyperparameter Tuning — Choosing the Best k

One of the key hyperparameters in KNN is **k** (the number of neighbors used to make a prediction). A small k is sensitive to noise; a large k oversimplifies. We loop over a range of k values and compare test accuracy to find the sweet spot.

In [ ]:
```
# Hyperparameter Tuning: loop over k values 1-20
k_values = range(1, 21)
train_scores = []
test_scores  = []

for k in k_values:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    knn_k.fit(X_train, y_train)
    train_scores.append(knn_k.score(X_train, y_train))
    test_scores.append(knn_k.score(X_test_imputed, y_test))

plt.figure(figsize=(9, 4))
plt.plot(k_values, train_scores, label='Train Accuracy', marker='o')
plt.plot(k_values, test_scores,  label='Test Accuracy',  marker='s')
plt.xlabel('k (Number of Neighbors)')
plt.ylabel('Accuracy')
plt.title('KNN: Accuracy vs. k')
plt.legend()
plt.tight_layout()
plt.show()

best_k = k_values[test_scores.index(max(test_scores))]
print(f'Best k on test set: {best_k}  (Test Accuracy = {max(test_scores):.3f})')
```




```
Best k on test set: 6  (Test Accuracy = 0.643)
```

In [ ]:
```python
# Retrain KNN with best k and print full evaluation
knn_best = KNeighborsClassifier(n_neighbors=best_k)
knn_best.fit(X_train, y_train)

knn_preds = knn_best.predict(X_test_imputed)
knn_train_score = knn_best.score(X_train, y_train)
knn_test_score  = knn_best.score(X_test_imputed, y_test)

print(f'KNN Train Accuracy (k={best_k}): {knn_train_score:.3f}')
print(f'KNN Test  Accuracy (k={best_k}): {knn_test_score:.3f}')
print('\nClassification Report (KNN Best k):')
print(classification_report(y_test, knn_preds))
```

```
KNN Train Accuracy (k=6): 0.589
KNN Test  Accuracy (k=6): 0.643

Classification Report (KNN Best k):
              precision    recall  f1-score   support

         Bad       0.00      0.00      0.00         1
        Good       0.58      1.00      0.74         7
     Neutral       1.00      0.33      0.50         6

    accuracy                           0.64        14
   macro avg       0.53      0.44      0.41        14
weighted avg       0.72      0.64      0.58        14
```

```
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Part 3.3: LINEAR REGRESSION MODEL

We chose Linear Regression as it can predict numerical values from other numerical values. Linear Regression implements a line of best fit to determine the outcomes

LR Pros

- Simple to Implement
- Good Predictive Adaptibility
- Efficiency

LR Cons

- Underfit risk
- Bad with Categorical data
- Over Simplistic

In [ ]:
```
#Importing LR
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import LabelEncoder
```

In [ ]:
```
df_subset = df_subset.astype(float)
```

In [ ]:
```
df_subset
```

Out[ ]:

	Gamer?	Social Media Hours	Video Game Hours	Anxiety Levels	Comparision	Self Esteem	Using for Relief	Lonliness	Sleep Affected	Addiction Levels	Daily Mood
0	2.0	2.0	0.0	2.0	2.0	3.0	3.0	3.0	3.0	4.0	2.0
1	0.0	3.0	0.0	4.0	2.0	2.0	2.0	3.0	3.0	2.0	2.0
2	0.0	2.0	0.0	2.0	1.0	2.0	3.0	1.0	4.0	2.0	3.0
4	2.0	3.0	2.0	1.0	2.0	1.0	3.0	2.0	3.0	3.0	3.0
5	1.0	2.0	0.0	2.0	3.0	2.0	4.0	2.0	1.0	2.0	2.0
...	...	...	...	...	...	...	...	...	...	...	...
95	2.0	1.0	1.0	4.0	4.0	5.0	4.0	5.0	4.0	5.0	4.0
96	0.0	1.0	0.0	5.0	5.0	2.0	2.0	5.0	3.0	4.0	4.0
97	1.0	3.0	1.0	3.0	4.0	5.0	4.0	5.0	4.0	4.0	4.0
98	1.0	1.0	3.0	3.0	3.0	1.0	4.0	4.0	5.0	3.0	2.0
99	2.0	2.0	3.0	5.0	5.0	5.0	5.0	5.0	5.0	5.0	0.0

70 rows × 11 columns

In [ ]:
```
le = LabelEncoder()
```

In [ ]:
```
y_train_encoded = le.fit_transform(y_train)
```

In [ ]:
```
y_test_encoded = le.fit_transform(y_test)
```

In [ ]:
```
LR = LinearRegression()
```

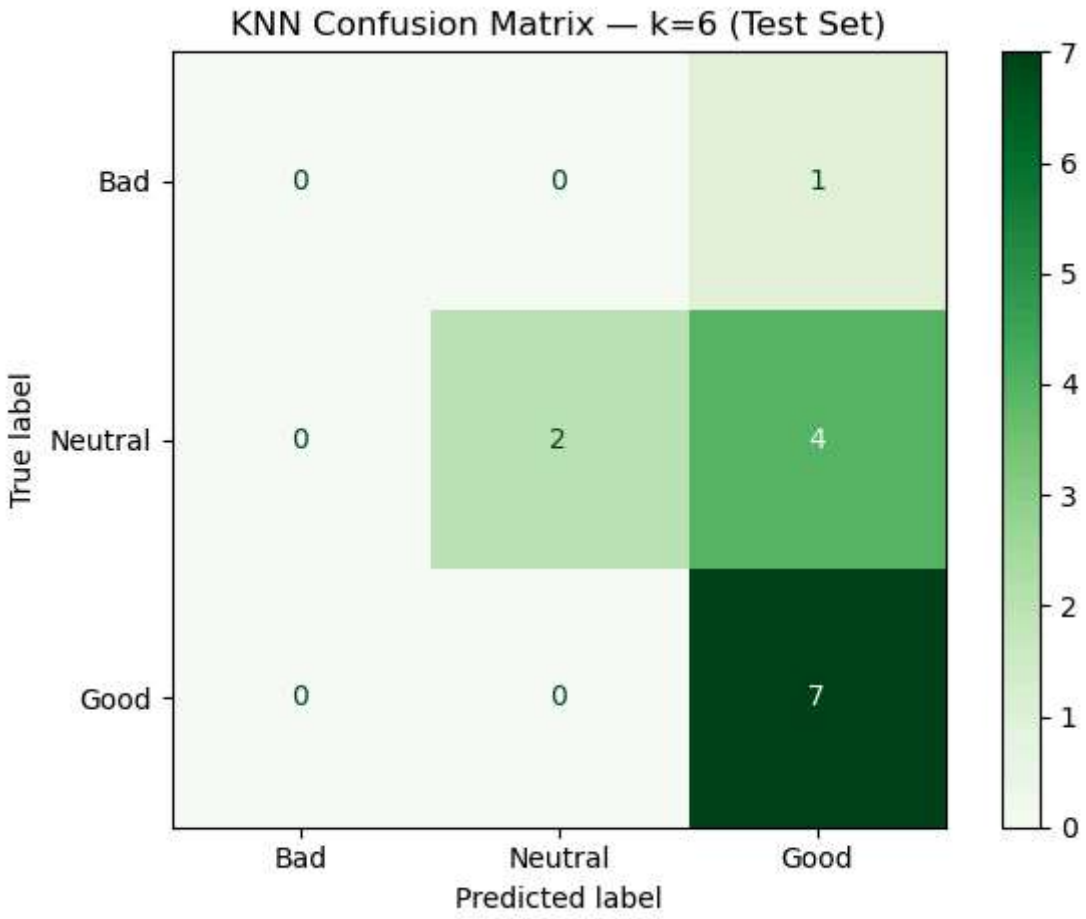
In [ ]:
```
LR.fit(X_train, y_train_encoded)
```

Out[ ]:
```
LinearRegression
LinearRegression()
```

In [ ]:
```
# Use the imputed test set (created earlier as X_test_imputed) to avoid NaNs
LR.score(X_test_imputed, y_test_encoded)
```

Out[ ]:
```
-0.2384063167120063
```

In [ ]:
```
# Confusion Matrix — KNN (Best k)
cm_knn = confusion_matrix(y_test, knn_preds, labels=['Bad', 'Neutral', 'Good'])
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_knn, display_labels=['Bad', 'Neutral', 'Good'])
disp2.plot(cmap='Greens')
plt.title(f'KNN Confusion Matrix — k={best_k} (Test Set)')
plt.tight_layout()
plt.show()
```



Part 3.4: Probability Calibration — Beyond Class Method

What is Probability Calibration?

Standard classifiers (like SVM) output a predicted *class* (e.g., 'Good', 'Neutral', 'Bad'), but not a meaningful *confidence score*. Probability Calibration wraps a classifier so that it also outputs a reliable probability. For example, if the model says there is a 70% chance someone is in a 'Bad' mood, we want that to actually be true 70% of the time across many predictions.

We use `CalibratedClassifierCV` with *isotonic regression*, which works by fitting a monotone function on the raw SVM outputs to map them to well-calibrated probabilities. This technique was **not covered in class** and represents additional depth in our

experimentation.

Why does this matter for mental health? Because knowing *how confident* the model is allows a user or a counselor to better judge whether to act on a prediction — a model that says '51% likely Bad' should be treated very differently from one that says '95% likely Bad'.

In [ ]:
```
# Part 3.4 — Probability Calibration (beyond class)
from sklearn.calibration import CalibratedClassifierCV, CalibrationDisplay

# Wrap the SVM in a calibration layer using isotonic regression (5-fold CV)
base_svc = SVC(class_weight='balanced', probability=False)  # base model
calibrated_model = CalibratedClassifierCV(base_svc, cv=5, method='isotonic')
calibrated_model.fit(X_train, y_train)

# Predict classes and probabilities on test set
cal_preds  = calibrated_model.predict(X_test_imputed)
cal_probas = calibrated_model.predict_proba(X_test_imputed)

cal_test_score = calibrated_model.score(X_test_imputed, y_test)
print(f'Calibrated SVM Test Accuracy: {cal_test_score:.3f}')
print('\nClassification Report (Calibrated SVM):')
print(classification_report(y_test, cal_preds))

# Show the probability output for first 10 test samples
classes = calibrated_model.classes_
print('\nSample Predicted Probabilities (first 10 rows):')
print(f'{"Class":>10}', '  '.join(f'{c:>9}' for c in classes))
for i, row in enumerate(cal_probas[:10]):
    print(f'  Sample {i+1}: ', '  '.join(f'{p:9.3f}' for p in row))
```

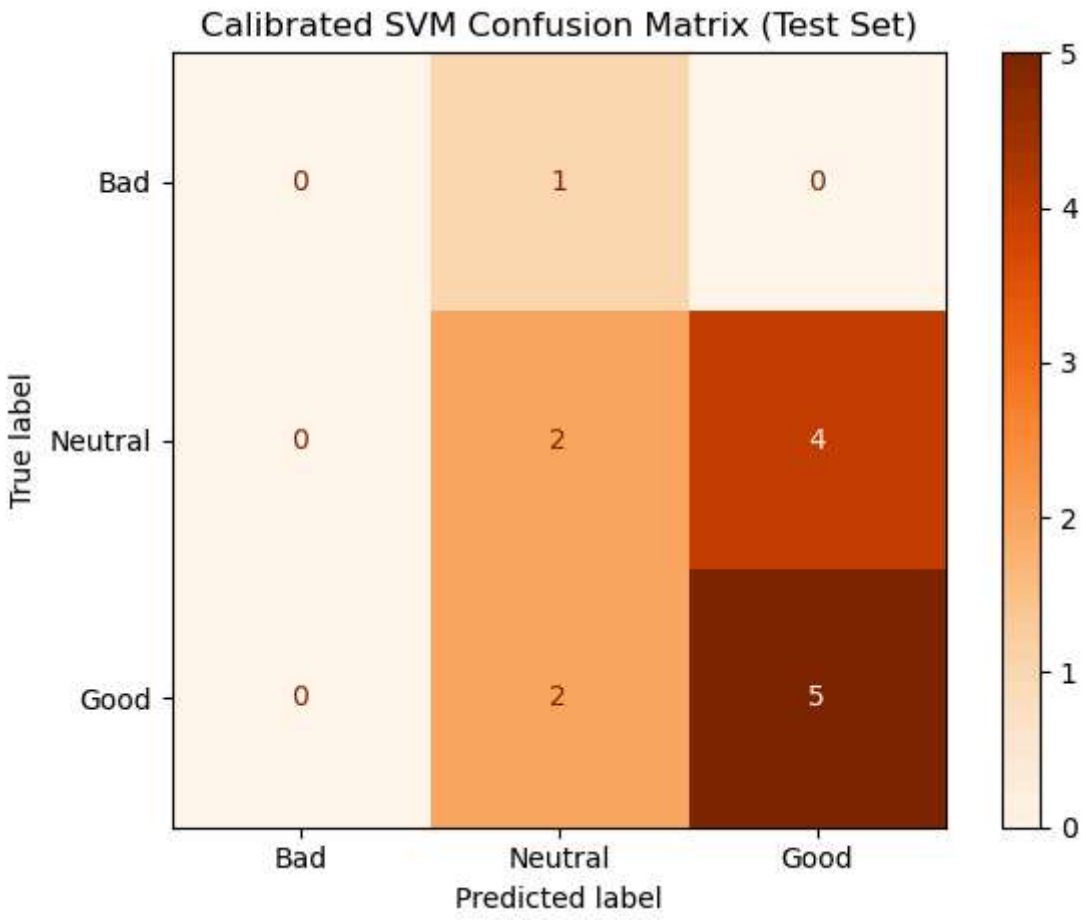
```
Calibrated SVM Test Accuracy: 0.500

Classification Report (Calibrated SVM):
              precision    recall  f1-score   support

         Bad       0.00      0.00      0.00         1
        Good       0.56      0.71      0.62         7
     Neutral       0.40      0.33      0.36         6

    accuracy                           0.50        14
   macro avg       0.32      0.35      0.33        14
weighted avg       0.45      0.50      0.47        14


Sample Predicted Probabilities (first 10 rows):
     Class       Bad       Good    Neutral
  Sample 1:      0.028      0.536      0.436
  Sample 2:      0.148      0.494      0.358
  Sample 3:      0.122      0.511      0.368
  Sample 4:      0.033      0.552      0.415
  Sample 5:      0.016      0.562      0.422
  Sample 6:      0.103      0.517      0.380
  Sample 7:      0.185      0.374      0.441
  Sample 8:      0.148      0.495      0.357
  Sample 9:      0.027      0.440      0.533
  Sample 10:      0.140      0.309      0.551
```

```
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precisio
n is ill-defined and being set to 0.0 in labels with no predicted samples. Use `zero_division` parameter to control t
his behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

In [ ]:

```
# Confusion Matrix – Calibrated SVM
cm_cal = confusion_matrix(y_test, cal_preds, labels=['Bad', 'Neutral', 'Good'])
disp3 = ConfusionMatrixDisplay(confusion_matrix=cm_cal, display_labels=['Bad', 'Neutral', 'Good'])
```

```
disp3.plot(cmap='Oranges')
plt.title('Calibrated SVM Confusion Matrix (Test Set)')
plt.tight_layout()
plt.show()
```



Part 3.5: Feature Selection Experiment

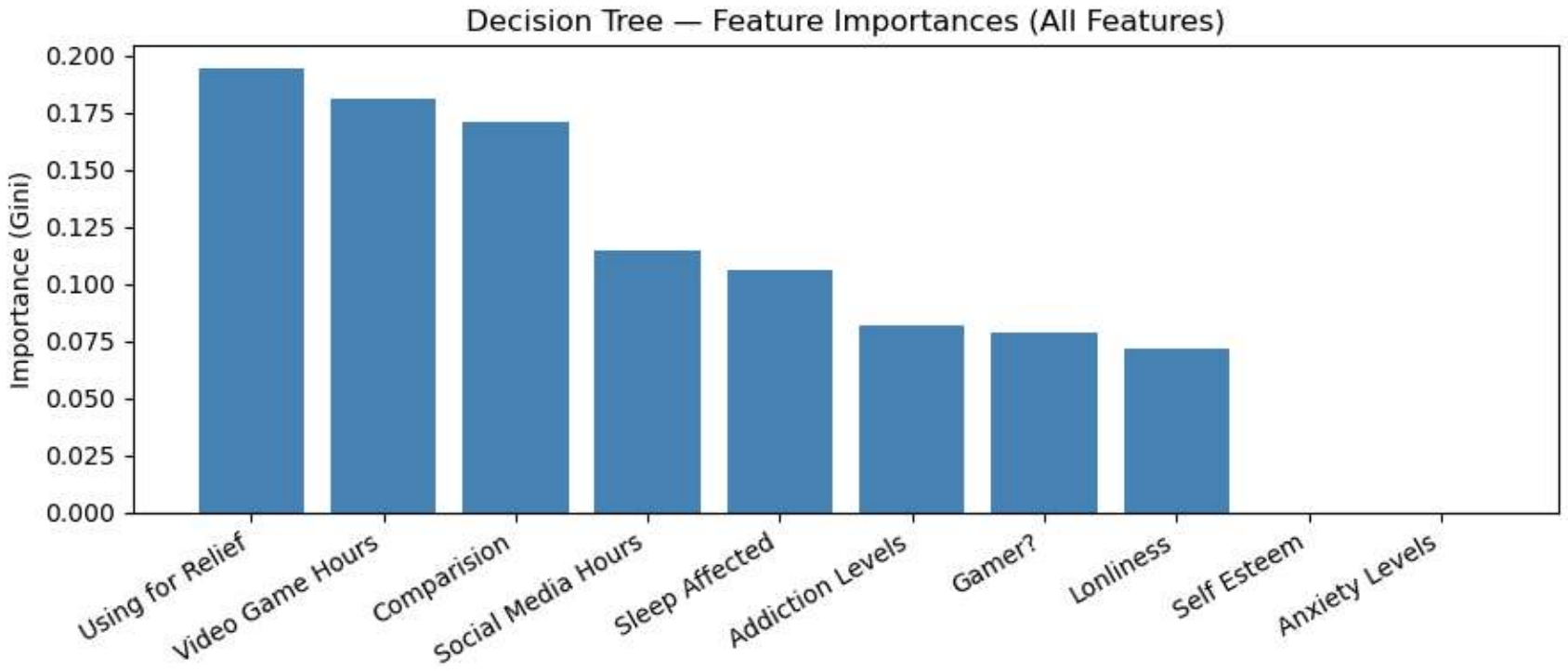
Which features actually matter most for predicting daily mood? We run two experiments:

1. **All features** — use every column in our model dataset.

2. **Selected features** — use only the most intuitively relevant ones: `Social Media Hours`, `Video Game Hours`, `Anxiety Levels`, `Sleep Affected`, and `Addiction Levels`.

We use a **Decision Tree** for this experiment because decision trees naturally rank features by how much they reduce impurity (Gini index), giving us a built-in importance measure. We imported `DecisionTreeClassifier` at the start of the notebook for exactly this purpose.

In [ ]:
```
# Feature Importance Experiment — Decision Tree
from sklearn.tree import DecisionTreeClassifier

# Experiment A: All Features
dt_all = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_all.fit(X_train, y_train)
score_all = dt_all.score(X_test_imputed, y_test)

# Feature importance plot — all features
importances = dt_all.feature_importances_
feat_names  = X_train.columns.tolist()
sorted_idx  = importances.argsort()[::-1]

plt.figure(figsize=(9, 4))
plt.bar([feat_names[i] for i in sorted_idx], importances[sorted_idx], color='steelblue')
plt.xticks(rotation=30, ha='right')
plt.title('Decision Tree — Feature Importances (All Features)')
plt.ylabel('Importance (Gini)')
plt.tight_layout()
plt.show()
print(f'Decision Tree (All Features) Test Accuracy: {score_all:.3f}')
```




```
Decision Tree (All Features) Test Accuracy: 0.571
```

In [ ]:
```python
# Experiment B: Selected Features only
selected_features = ['Social Media Hours', 'Video Game Hours', 'Anxiety Levels',
                     'Sleep Affected', 'Addiction Levels']

X_sel_train = X_train[selected_features]
X_sel_test  = X_test_imputed[selected_features]

dt_sel = DecisionTreeClassifier(max_depth=4, random_state=42)
dt_sel.fit(X_sel_train, y_train)
score_sel = dt_sel.score(X_sel_test, y_test)

print(f'Decision Tree (All Features)      Test Accuracy: {score_all:.3f}')
print(f'Decision Tree (Selected Features) Test Accuracy: {score_sel:.3f}')
print('\nConclusion: Using only screen-time / mental health features',
      'vs. all features shows whether demographic columns add noise or signal.')
```

```
Decision Tree (All Features)      Test Accuracy: 0.571
Decision Tree (Selected Features) Test Accuracy: 0.429

Conclusion: Using only screen-time / mental health features vs. all features shows whether demographic columns add no
ise or signal.
```

Part 4: Model Comparison — Summary Table

Below we consolidate the test accuracy of every model we ran so we can directly compare performance. We also note what each model's key strength is and why we included it.

In [ ]:
```python
# Model Comparison Summary
import warnings
warnings.filterwarnings('ignore')

# Re-compute all test scores cleanly in one place
lr_test = LR.score(X_test_imputed, y_test_encoded)

results = {
    'Model':          ['SVM (default)', f'KNN (k={best_k})',  'Calibrated SVM', 'DT (All Features)', 'DT (Selected)
    'Train Accuracy':  [svc_train_score, knn_train_score, calibrated_model.score(X_train, y_train),
                        dt_all.score(X_train, y_train), dt_sel.score(X_sel_train, y_train)],
    'Test Accuracy':   [svc_test_score,  knn_test_score,  cal_test_score,
                        score_all, score_sel],
    'Type':           ['Classification', 'Classification', 'Classification (Calibrated)',
                        'Classification', 'Classification'],
}

results_df = pd.DataFrame(results)
results_df['Train Accuracy'] = results_df['Train Accuracy'].round(3)
results_df['Test Accuracy']  = results_df['Test Accuracy'].round(3)
print(results_df.to_string(index=False))

# Bar chart comparison
x = range(len(results['Model']))
width = 0.35
fig, ax = plt.subplots(figsize=(10, 5))
ax.bar([i - width/2 for i in x], results['Train Accuracy'], width, label='Train', color='cornflowerblue')
ax.bar([i + width/2 for i in x], results['Test Accuracy'],  width, label='Test',  color='coral')
```

```
ax.set_xticks(list(x))
ax.set_xticklabels(results['Model'], rotation=15, ha='right')
ax.set_ylabel('Accuracy')
ax.set_title('Model Comparison — Train vs. Test Accuracy')
ax.legend()
plt.tight_layout()
plt.show()
```

Model	Train Accuracy	Test Accuracy	Type
SVM (default)	0.625	0.500	Classification
KNN (k=6)	0.589	0.643	Classification
Calibrated SVM	0.679	0.500	Classification (Calibrated)
DT (All Features)	0.768	0.571	Classification
DT (Selected)	0.625	0.429	Classification